📘 PRODUCT DESIGN DOCUMENT (PDD)

🎓 Project: University Management System (MERN)

---

1. Product Overview

📌 Product Name:

EduManage (Working Name)

🎯 Objective:

Build a **role-based university management system** where:

- Admin manages entire system
- Students access academic data
- Teachers manage classes & evaluation

👥 2. User Roles & Permissions (RBAC)

👑 Admin

- Full access
- CRUD everything
- Manage roles

👨‍🏫 Teacher

- Manage courses
- Attendance
- Assignments
- Marks

👨‍🎓 Student

- View courses
- Attendance
- Results
- Submit assignments

---

3. Core Modules (Feature Breakdown)

🔐 3.1 Authentication Module

- Login (Admin / Teacher / Student)
- JWT-based authentication
- Role-based access middleware
- Forgot/reset password

3.2 User Management

- Create users (Admin)
- Assign roles
- Profile management

3.3 Student Management

- Add/edit/delete students
- Assign course/batch
- Student profile
- Academic history

3.4 Course Management

- Create course
- Add subjects
- Assign teacher
- Assign students

3.5 Attendance System

- Mark attendance (Teacher)
- View attendance (Student/Admin)
- Generate reports

3.6 Examination System

- Create exams
- Assign subjects
- Enter marks
- Publish results

3.7 Result Module

- Generate marksheet
- GPA/percentage calculation
- Student performance analytics

3.8 Assignment Module

- Upload assignments (Teacher)
- Submit assignments (Student)
- Track submissions

💰 3.9 Fees Module

- Fee structure
- Payment tracking
- Payment history

📢 3.10 Notification System

- Admin → broadcast messages
- Role-based notifications

🎥 3.11 Study Materials

- Upload PDFs/videos
- Download by students

⚙ 3.12 Settings & Roles

- Role management
- Permissions control
- System configs

□ 4. Complete Page Structure (Frontend - React)

🔐 Auth Pages

- `/login`
- `/forgot-password`
- `/reset-password`

🏠 Dashboard

- `/dashboard`
- Stats cards
- Charts (students, revenue, etc.)

👤 Profile

- `/profile`

🎓 Students

- `/students`
- `/students/add`
- `/students/:id`

📚 Courses

- `/courses`
- `/courses/:id`

📅 Attendance

- `/attendance`

📝 Exams

- `/exams`

📊 Results

- `/results`

📂 Assignments

- `/assignments`

💰 Fees

- `/fees`

📢 Notifications

- `/notifications`

⚙ Admin

- `/roles`
- `/settings`

🔄 5. User Flow (End-to-End)

□🎓 Student Flow

```
Login →
Dashboard →
View Courses →
Check Attendance →
Download Material →
Submit Assignment →
Check Result →
Pay Fees
```

👨‍🏫 Teacher Flow

```
Login →
Dashboard →
View Courses →
Mark Attendance →
Upload Assignment →
Enter Marks
```

👑 Admin Flow

```
Login →
Dashboard →
Add Course →
Add Student →
Assign Teacher →
Monitor System →
Generate Reports
```

🛠 6. Backend Architecture (Node + Express)

📁 Folder Structure

```
src/
 ├── controllers/
 ├── models/
 ├── routes/
 ├── middleware/
 ├── utils/
 ├── config/
 └── server.js
```

🔌 API Structure

Auth

```
POST   /api/auth/login
POST   /api/auth/register
POST   /api/auth/forgot-password
```

Users

```
GET    /api/users
```

```
POST   /api/users
PUT    /api/users/:id
DELETE /api/users/:id
```

Students

```
GET    /api/students
POST   /api/students
GET    /api/students/:id
```

Courses

```
POST   /api/courses
GET    /api/courses
```

Attendance

```
POST   /api/attendance
GET    /api/attendance/:studentId
```

Exams

```
POST   /api/exams
POST   /api/results
GET    /api/results/:studentId
```

□ 7. Database Design (MongoDB)

👤 Users Schema

```
{
  name,
  email,
  password,
  role, // admin, teacher, student
}
```

🎓 Students

```
{
  userId,
  courseId,
  semester,
  attendance,
}
```

📚 Courses

```
{
  name,
  subjects,
  teacherId
}
```

📅 Attendance

```
{
  studentId,
  subject,
  date,
  status
}
```

📝 Results

```
{
  studentId,
  subject,
  marks,
  grade
}
```

📂 Assignments

```
{
  title,
  fileUrl,
  studentId,
  status
}
```

💰 Fees

```
{
  studentId,
  amount,
  status,
  paymentDate
}
```

🧠 8. Frontend Architecture (React)

📁 Folder Structure

```
src/
 ├── components/
 ├── pages/
 ├── hooks/
 ├── services/ (API calls)
 ├── context/ (Auth)
 ├── utils/
```

⚛ Key Features

- Protected Routes
- Role-based UI
- Axios API integration
- Global state (Context API)

🔐 9. Security

- JWT Authentication
- Password hashing (bcrypt)
- Role-based middleware
- Input validation
- Rate limiting

⚡ 10. Performance Optimization

- Pagination
- Lazy loading
- Caching (optional Redis)
- Optimized queries

🚀 11. Deployment

Frontend:

- Netlify / Vercel

Backend:

- Render

Database:

- MongoDB Atlas

📊 12. Future Enhancements

- Live classes integration
- Video lectures
- Mobile app
- Analytics dashboard

💡 13. MVP (Minimum Viable Product)

👉 Build THIS first:

- Auth (JWT)
- Student CRUD
- Course CRUD
- Attendance
- Results

□ 14. Real Industry Insight

👉 This project =
ERP + LMS (Learning Management System)

🔥 **FINAL MENTOR NOTE**

If you build this:

✅ You cover:

- Full MERN stack
- Real SaaS architecture
- RBAC system
- Production-level logic